Atomic type	Corresponding specialization
atomic_ulong	std::atomic<unsigned long>
atomic_llong	std::atomic<long long>
atomic_ullong	std::atomic<unsigned long long>
atomic_char16_t	std::atomic<char16_t>
atomic_char32_t	std::atomic<char32_t>
atomic_wchar_t	std::atomic<wchar_t>

As well as the basic atomic types, the C++ Standard Library also provides a set of `typedef`s for the atomic types corresponding to the various nonatomic Standard Library `typedef`s such as `std::size_t`. These are shown in table 5.2.

That's a lot of types! There's a rather simple pattern to it; for a standard `typedef T`, the corresponding atomic type is the same name with an `atomic_` prefix: `atomic_T`. The same applies to the built-in types, except that `signed` is abbreviated as just `s`, `unsigned` as just `u`, and `long long` as `llong`. It's generally just simpler to say `std::atomic<T>` for whichever `T` you wish to work with, rather than use the alternative names.

Table 5.2. The standard atomic typedefs and their corresponding built-in typedefs

Atomic typedef	Corresponding Standard Library typedef
atomic_int_least8_t	int_least8_t
atomic_uint_least8_t	uint_least8_t
atomic_int_least16_t	int_least16_t
atomic_uint_least16_t	uint_least16_t
atomic_int_least32_t	int_least32_t
atomic_uint_least32_t	uint_least32_t
atomic_int_least64_t	int_least64_t
atomic_uint_least64_t	uint_least64_t
atomic_int_fast8_t	int_fast8_t
atomic_uint_fast8_t	uint_fast8_t
atomic_int_fast16_t	int_fast16_t
atomic_uint_fast16_t	uint_fast16_t
atomic_int_fast32_t	int_fast32_t
atomic_uint_fast32_t	uint_fast32_t
atomic_int_fast64_t	int_fast64_t
atomic_uint_fast64_t	uint_fast64_t
atomic_intptr_t	intptr_t
atomic_uintptr_t	uintptr_t
atomic_size_t	size_t
atomic_ptrdiff_t	ptrdiff_t
atomic_intmax_t	intmax_t
atomic_uintmax_t	uintmax_t

The standard atomic types are not copyable or assignable in the conventional sense, in that they have no copy constructors or copy assignment operators. They *do*, however, support assignment from and implicit conversion to the corresponding built-in types as well as direct `load()` and